Komputasi Paralel dan Distributed — Dask

Pertemuan 7 — Analisis Big Data

S1 Teknik Informatika / Sistem Informasi | Semester Ganjil 2026

Dosen Pengampu: Yuda Munarko

Agenda Perkuliahan Hari Ini

1. **Fondasi Komputasi Paralel & Tantangan Skala Besar**
   - Paradigma *Scale-Up* vs *Scale-Out*, keterbatasan RAM (*Out-Of-Memory*), dan hambatan Python GIL.
2. **Arsitektur Dask: Dari Abstraksi Koleksi hingga Eksekusi Paralel**
   - 4 Lapisan Dask: *Collections*, *Task Graph (DAG)*, *Schedulers*, dan *Worker Nodes*.
3. **Komparasi Fundamental: Dask vs Apache Spark**
   - Runtime Python-native vs JVM, *Catalyst Optimizer* vs *Dynamic DAG*, dan panduan pemilihan teknologi.
4. **Koleksi Dask:** `dask.delayed` & `dask.dataframe`
   - Memparalelkan fungsi kustom, partisi data out-of-core, eksekusi malas (*lazy execution*), serta peran `.compute()` & `.persist()`.
5. **Observabilitas Real-time: Dask Web Dashboard (Port 8787)**
   - Membaca *Task Stream* (Gantt chart), *Progress Bar*, dan ambang batas *Worker Memory Spilling*.
6. **Strategi Partisi, Diagnostik Masalah Terdistribusi, & Check-in Tugas 2 Phase 1**
   - Aturan emas partisi 100MB, penanganan error `KilledWorker`, serta evaluasi milestone tim.

> **Sesi Interaktif Wayground**: Kuis dibagi dalam 3 Checkpoint (Menit 25, Menit 55, dan Menit 85).

Mengapa Kita Butuh Komputasi Paralel & Terdistribusi?

Pada pertemuan 2 hingga 6, kita menggunakan **Polars** dan **DuckDB** yang sangat cepat memanfaatkan seluruh *core* CPU pada satu laptop/server (*single-node multithreading*).

Batasan Single-Node (Vertikal / Scale-Up)
Meningkatkan kemampuan satu komputer dengan menambah RAM (misal 16GB ke 64GB) dan CPU cores.

Solusi Multi-Node (Horizontal / Scale-Out)
Membagi beban kerja ke dalam kluster yang terdiri dari puluhan atau ratusan mesin yang bekerja sama.

Tembok Keterbatasan Single-Node:

- **Batasan Fisik RAM:** Dataset e-commerce nasional atau sensor multi-tahun dapat mencapai ratusan gigabyte hingga terabyte.
- **Harga Eksponensial:** Server dengan RAM 1TB jauh lebih mahal dibanding kumpulan server komoditas.
- **Single Point of Failure:** Jika satu mesin gagal, seluruh pemrosesan runtuh.

Karakteristik Komputasi Terdistribusi:

- **Partisi Data:** Data dipecah menjadi ribuan blok kecil (*chunks*).
- **Eksekusi Paralel:** Setiap *worker* memproses partisi lokal miliknya sendiri secara mandiri.
- **Toleransi Kesalahan (*Fault Tolerance*):** Tugas dapat diulang jika satu *worker* mati.

Tantangan Spesifik Python: Global Interpreter Lock (GIL)

Sebelum memahami Dask, kita harus memahami mengapa *multithreading* Python standar sering kali tidak mempercepat perhitungan CPU murni.

> **Definisi: Python GIL (Global Interpreter Lock)**
> Mekanisme internal pada CPython (implementasi standar Python) yang memastikan hanya **satu thread Python** yang dapat mengeksekusi bytecode pada satu waktu, meskipun komputer memiliki 16 core CPU.

Dampak GIL:

- Operasi I/O (baca file disk, request HTTP) melepaskan GIL (bisa paralel).
- Operasi CPU murni (loop Python) terhambat oleh GIL (kecepatan sama dengan 1 core).

Bagaimana Dask Mengatasi GIL?

1. **Multiprocessing Scheduler:**
   - Dask menjalankan banyak proses Python terpisah yang masing-masing memiliki memori dan GIL sendiri.
2. **Library C/Rust yang Melepaskan GIL:**
   - Saat Dask mengeksekusi kode NumPy, Pandas, Scipy, atau PyArrow, library tersebut melepaskan GIL selama komputasi numerik C/C++.
3. **Dask Distributed Scheduler:**
   - Mengatur pekerja di level sistem operasi bahkan lintas mesin jaringan dengan transfer data berbasis soket TCP.

Konsep Kunci: Directed Acyclic Graph (DAG)

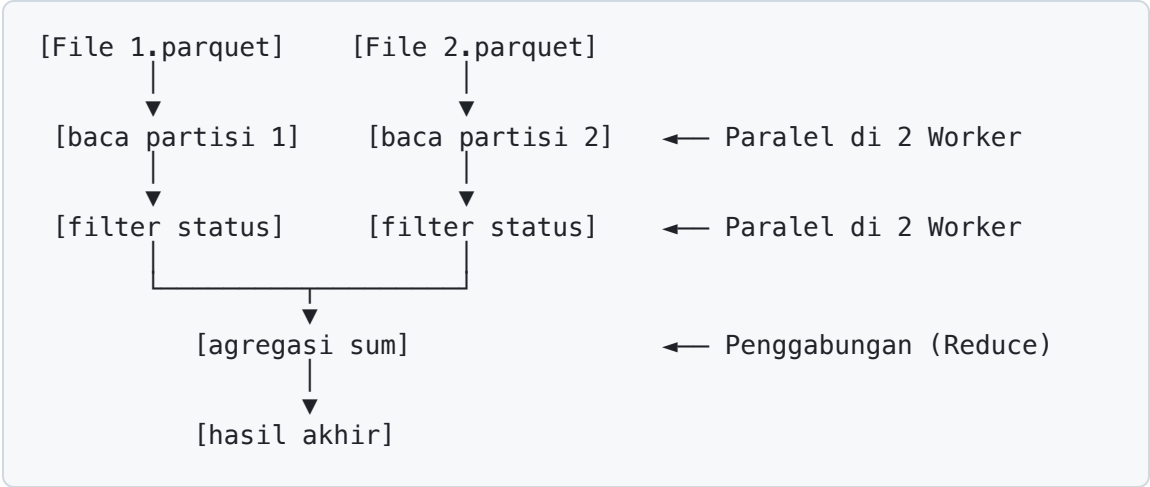
Seluruh komputasi di Dask tidak langsung dieksekusi saat kode ditulis, melainkan disusun ke dalam representasi matematika yang disebut **DAG** (*Task Graph*).

Konsep: Directed Acyclic Graph (DAG)

Grafik terarah tanpa siklus tertutup di mana setiap **simpul (node)** merepresentasikan fungsi atau operasi data, dan setiap **sisi (edge)** merepresentasikan aliran ketergantungan data antar operasi.

Mengapa DAG Penting?

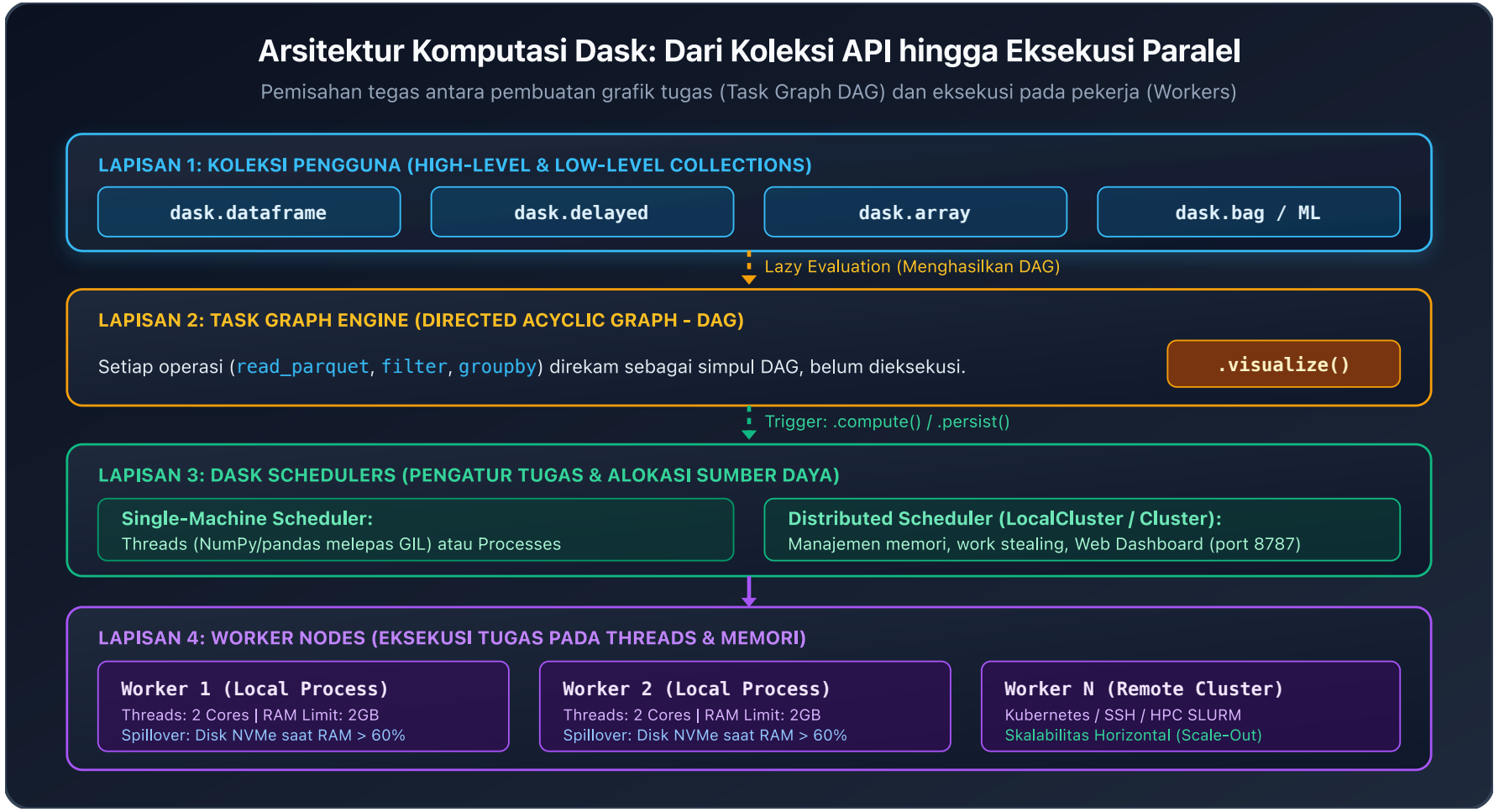
- **Optimasi Sebelum Eksekusi:** Dask dapat menghapus langkah yang tidak diperlukan (*dead code elimination*) dan menyatukan filter (*culling*).
- **Paralelisasi Otomatis:** Operasi yang tidak saling bergantung pada tingkat percabangan DAG langsung dikirim ke *worker* yang berbeda secara bersamaan.



Dask membuat DAG secara instan (fraksi milidetik) tanpa menyentuh data disk sesungguhnya!

Ilustrasi Visual: Arsitektur 4 Lapisan Dask

Dask memisahkan antarmuka pengguna, representasi grafik tugas, alokasi penjadwalan, dan mesin pekerja.



Tautan Dokumentasi Resmi: Dask Architecture Overview

Lapisan Penjadwalan: Single-Machine vs Distributed

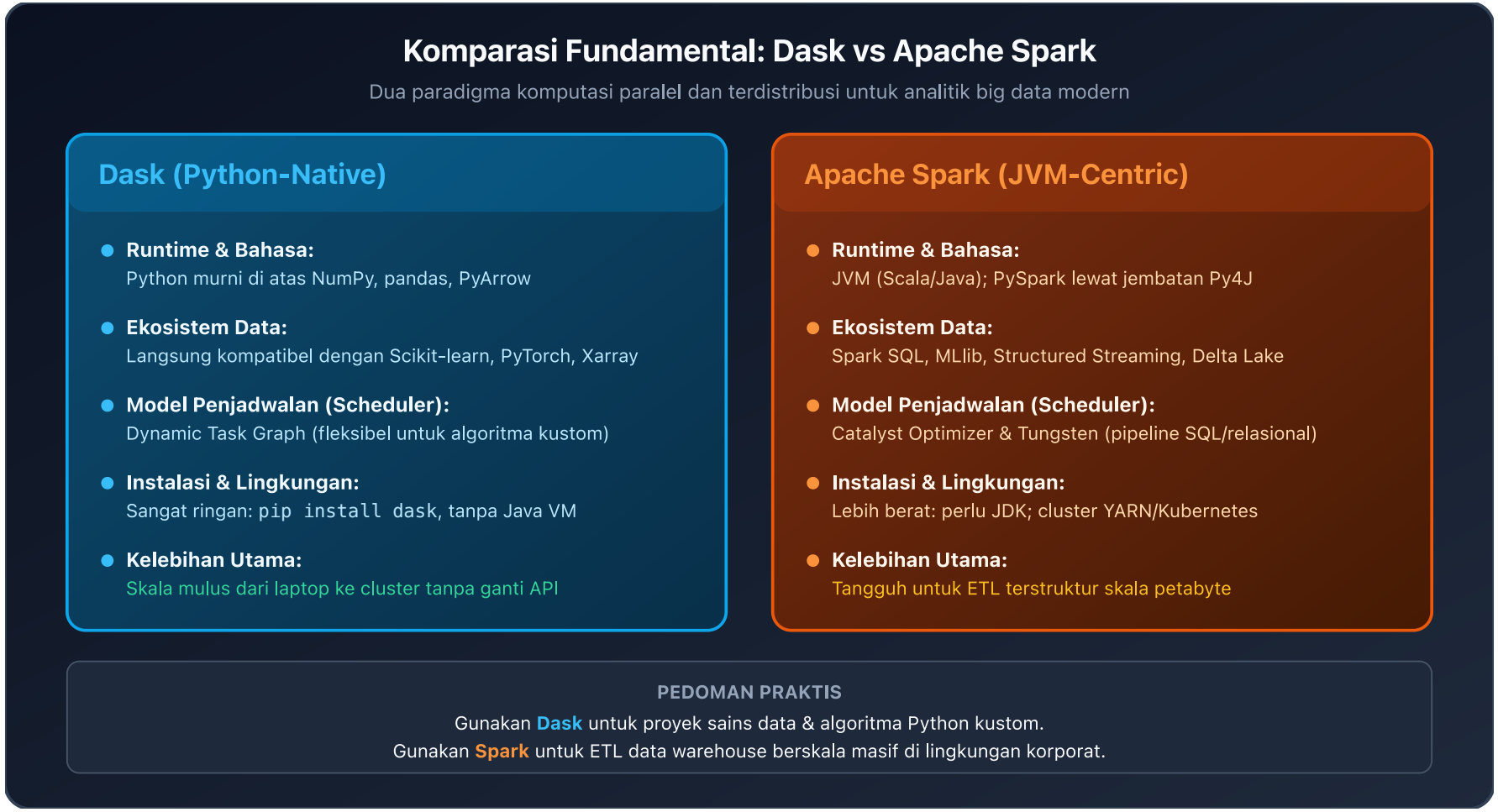
Dask memiliki dua jenis *scheduler* utama dengan karakteristik performa yang berbeda:

Fitur / Karakteristik	Single-Machine Schedulers	Dask Distributed Scheduler (`dask.distributed`)
Target Lingkungan	Laptop / Server 1 mesin	Laptop, Workstation multi-core, & Multi-node Cluster
Pilihan Mode	`threaded` atau `processes`	`LocalCluster` atau Cluster Remote (K8s, HPC, Cloud)
Web Dashboard	❌ Tidak tersedia	✅ Tersedia secara *live* pada port **8787**
Manajemen Memori	Sederhana, rawan OOM	Cerdas, otomatis *spill* data ke disk saat RAM kritis
Penanganan Kegagalan	Proses gugur jika crash	*Work stealing* & penugasan ulang tugas otomatis
Inisialisasi Kode	Otomatis default saat `.compute()`	`from dask.distributed import Client; client = Client()`

> **Rekomendasi Utama:** Selalu gunakan `dask.distributed.Client()` meskipun hanya bekerja di satu laptop, karena fitur **Web Dashboard** dan manajemen memori jauh lebih superior!

Komparasi Mendalam: Dask vs Apache Spark

Dua kerangka kerja terpopuler untuk memproses data skala besar. Kapan kita memilih Dask, dan kapan memilih Apache Spark?



Tautan Sumber: Dask and Apache Spark Comparison Guide

Matriks Pemilihan Alat: Polars vs DuckDB vs Dask vs Spark

Bagaimana memilih alat yang tepat untuk kebutuhan analitik di industri?

Skenario Kebutuhan	Ukuran Dataset	Pilihan Utama	Alasan Rasional Arsitektur
Eksplorasi Cepat & Waktu Nyata	< RAM (s.d. 30GB)	**Polars**	Single-node tercepat berkat Apache Arrow & Rust multithreading murni tanpa overhead cluster.
Analitik SQL & Hive Partition	S.d. 3x RAM laptop	**DuckDB**	Mesin SQL columnar *vectorized execution* terbaik dengan konsumsi memori sangat hemat.
Pipeline PyData & Algoritma Kustom	> RAM (s.d. Terabyte)	**Dask**	Skalabilitas paralel native Python yang kompatibel dengan NumPy, Pandas, Scikit-learn.
Data Lakehouse Korporat & BI SQL	Petabyte di Cloud	**Apache Spark**	Standar enterprise teruji dengan ekosistem Delta Lake, ACID, dan tim *data platform* besar.

> **Aturan Efisiensi:** *"Jangan gunakan komputasi terdistribusi jika data masih muat di RAM single-node!"* Overhead penjadwalan terdistribusi justru memperlambat data kecil.

Komponen 1: `dask.delayed` untuk Paralelisasi Kode Kustom

Bagaimana jika kita memiliki fungsi Python kustom (looping iterasi, pemrosesan citra, scraping web) yang tidak berbasis DataFrame tabular?

Definisi: dask.delayed

Dekorator atau fungsi pembungkus yang menunda eksekusi suatu fungsi Python biasa, sehingga pemanggilannya hanya menghasilkan simpul tugas di dalam DAG.

Pola Eksekusi Sekuensial (Lambat):

```
def ambil_data(url):
    time.sleep(1) # Simulasi I/O
    return len(url)

# Berjalan sekuensial: 1 + 1 + 1 = 3 detik!
hasil = [ambil_data(u) for u in daftar_url]
total = sum(hasil)
```

Pola Eksekusi Paralel dengan Delayed:

```
import dask

# Membungkus fungsi menjadi operasi lazy
ambil_delayed = dask.delayed(ambil_data)

# Tidak ada komputasi yang berjalan di sini!
# Hanya membuat 3 node delayed di Task Graph
tugas = [ambil_delayed(u) for u in daftar_url]
total_delayed = dask.delayed(sum)(tugas)

# Eksekusi paralel sesungguhnya: hanya ~1 detik!
hasil_akhir = total_delayed.compute()
```

Dask membagikan tugas `ambil_data` ke 3 worker berbeda secara serentak!

Anatomi Grafik Tugas Delayed: `.visualize()`

Salah satu fitur paling berharga untuk pembelajaran dan diagnostik adalah melihat grafik tugas secara grafis sebelum dieksekusi.

```python
import dask

@dask.delayed
def gandakan(x):
    return x * 2

@dask.delayed
def tambah(a, b):
    return a + b

# Membangun DAG percabangan
x = gandakan(10)
y = gandakan(20)
z = tambah(x, y)

# Render Task Graph ke file gambar SVG/PNG
z.visualize(filename="assets/delayed_dag.svg")
```

Cara Membaca Grafik Task:

- **Lingkaran (Circles):** Fungsi komputasi yang akan dipanggil (misal: `gandakan`, `tambah`).
- **Persegi (Rectangles):** Nilai data atau konstanta awal (misal: `10`, `20`).
- **Panah (Arrows):** Arah dependensi data; simpul anak baru dieksekusi setelah simpul induk selesai.

> **Prasyarat Sistem:** Membutuhkan utilitas sistem `graphviz` dan package `graphviz` di environment Docker.

Anti-Pattern dan Jebakan Penggunaan `dask.delayed`

`dask.delayed` sangat kuat, tetapi jika digunakan secara keliru justru akan memperlambat program.

Jebakan 1: Memparalelkan Fungsi Terlalu Cepat

Biaya overhead penjadwalan Dask berkisar **200–500 mikrodetik per tugas**. Jika fungsi yang di-delay hanya membutuhkan waktu 1 mikrodetik (misal: operasi `x + 1` biasa), overhead Dask akan 100x lebih lambat dibanding loop biasa!

Aturan: Hanya gunakan `delayed` untuk fungsi yang membutuhkan waktu **> 20 milidetik** (misal: pembacaan file, query database, komputasi numerik berat).

Jebakan 2: Memanggil .compute() di dalam Loop

```
# ❌ KELIRU: Kehilangan manfaat paralel!
hasil = []
for item in daftar_data:
    t = dask.delayed(proses)(item)
    hasil.append(t.compute()) # Eksekusi sekuensial!

# ✅ BENAR: Kumpulkan semua, compute sekali!
tugas = [dask.delayed(proses)(item) for item in daftar_data]
hasil = dask.compute(*tugas) # Paralel murni!
```

Komponen 2: `dask.dataframe` — Big Data Tabular

Bagaimana memproses file CSV atau Parquet berukuran puluhan gigabyte yang tidak muat ke dalam RAM laptop?

Definisi: dask.dataframe

Struktur data tabular out-of-core yang terdiri dari kumpulan banyak **Pandas DataFrame** yang dibagi per partisi baris. Meniru hampir seluruh API Pandas standar.

Karakteristik Kunci:

- **Lazy by Default:** Operasi `df.read_parquet()`, `df[df.nilai > 50]`, dan `df.groupby()` tidak menghitung data di memori sampai `.compute()` dipanggil.
- **Out-of-Core:** Mampu memproses data 100GB pada laptop RAM 8GB dengan cara membaca dan membuang partisi bergantian.

```python
import dask.dataframe as dd

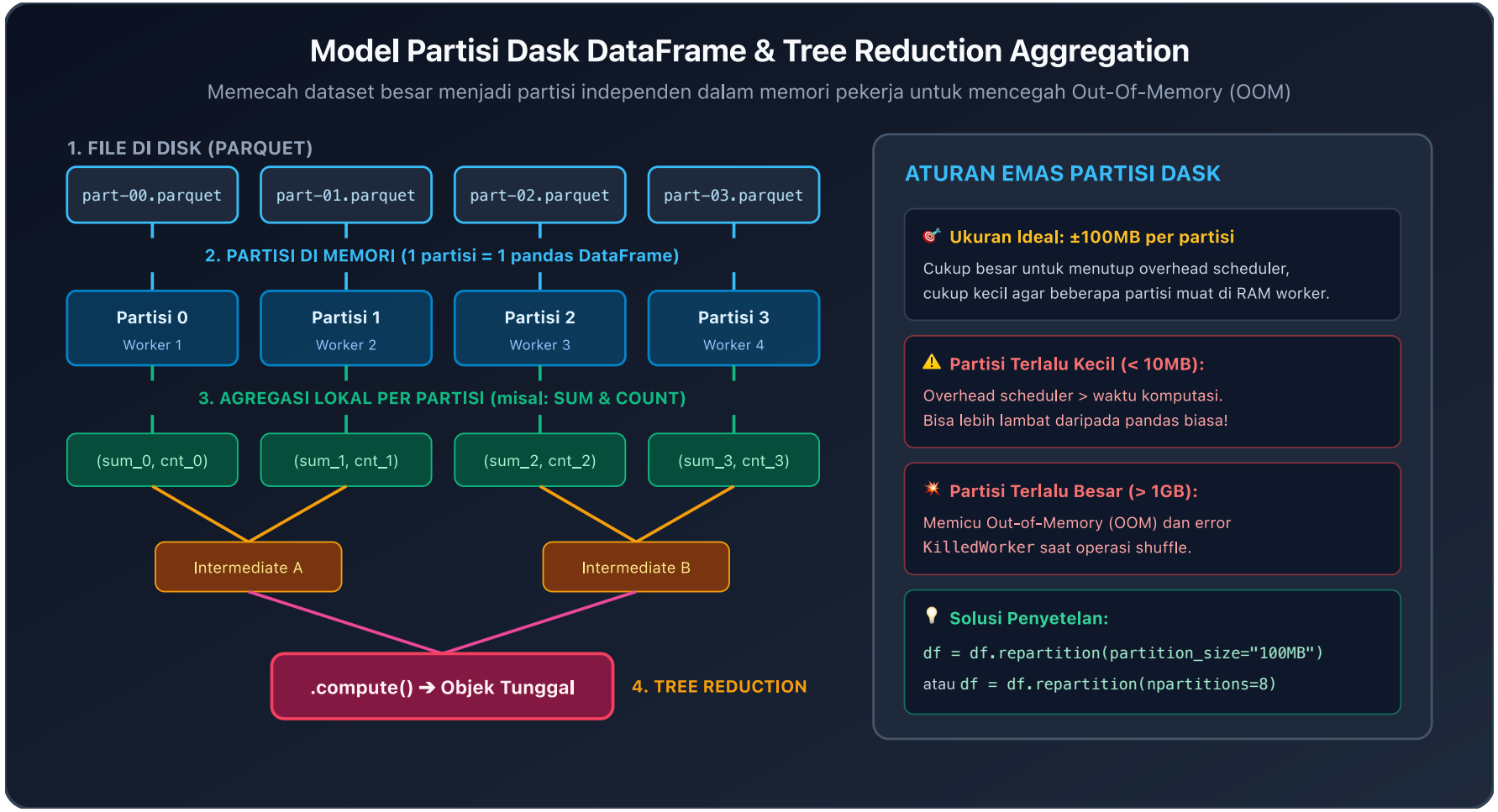
# Membaca seluruh file parquet dalam hitungan milidetik
# Data belum dibaca ke RAM! Hanya membaca skema metadata
ddf = dd.read_parquet(
    "data/transaksi/*.parquet",
    columns=["user_id", "kategori", "total_bayar"]
)

# Menulis ekspresi analitik lazily
ringkasan = (
    ddf[ddf["total_bayar"] > 50_000]
    .groupby("kategori")["total_bayar"]
    .agg(["count", "mean", "sum"])
)

# Memicu eksekusi nyata pada semua core pekerja
df_hasil = ringkasan.compute()
```

Ilustrasi Visual: Partisi & Tree Reduction Aggregation

Bagaimana Dask DataFrame memecah file masif menjadi partisi kecil dan mengagregasi hasilnya secara hierarkis?



Tautan Sumber: Dask DataFrame Partitioning Documentation

Operasi Mudah (Embarrassingly Parallel) vs Sulit (Shuffling)

Tidak semua operasi di Dask DataFrame memiliki efisiensi yang sama. Kenali dua kelompok operasi ini:

🟢 Operasi Partisi Lokal (Efisien)

Tidak memerlukan pertukaran data antar-pekerja (*Zero Data Shuffling*). Setiap *worker* bekerja 100% independen.

- **Filter Baris:** `ddf[ddf.status == 'SUCCESS']`
- **Transformasi Kolom:** `ddf['total'] * 1.11`
- **Menambah Kolom Turunan:** `ddf.assign(...)`
- **Drop Duplikat Kolom:** `ddf.drop(columns=[...])`
- **Operasi String Element-wise:** `ddf['email'].str.lower()`

Kecepatan penskalaan: **Linear sempurna (Ideal)**.

🔴 Operasi Shuffling (Biaya Jaringan Tinggi)

Membutuhkan penyusunan ulang baris data lintas seluruh *worker* melalui soket jaringan / disk.

- **Global Sorting:** `ddf.sort_values('waktu')`
- **Menetapkan Index Baru:** `ddf.set_index('user_id')`
- **Join Lintas Tabel:** `ddf.merge(ddf_lain, on='id')`
- **Groupby Cardinalitas Tinggi:** `ddf.groupby('user_uuid').sum()`

Peringatan: Operasi *shuffle* rawan memicu lonjakan memori dan latensi I/O. Lakukan *filtering* terlebih dahulu!

Dua Metode Eksekusi: Kapan Menggunakan `.compute()` vs `.persist()`?

Banyak data scientist pemula bingung membedakan antara `.compute()` dan `.persist()`.

Metode: .compute()
Mengeksekusi seluruh Task Graph, mengumpulkan **seluruh data hasil** dari pekerja, dan mengubahnya menjadi objek tunggal lokal di memori Python driver (Pandas DataFrame, NumPy array, atau skalar).

Kapan Menggunakan:

- Pada langkah **paling akhir** analisis ketika data hasil sudah teragregasi menjadi ukuran kecil (misal: tabel ringkasan 1.000 baris).
- ⚠️ **Bahaya:** Memanggil `.compute()` pada DataFrame 50 juta baris akan membuat laptop Anda *crash* / Out-Of-Memory!

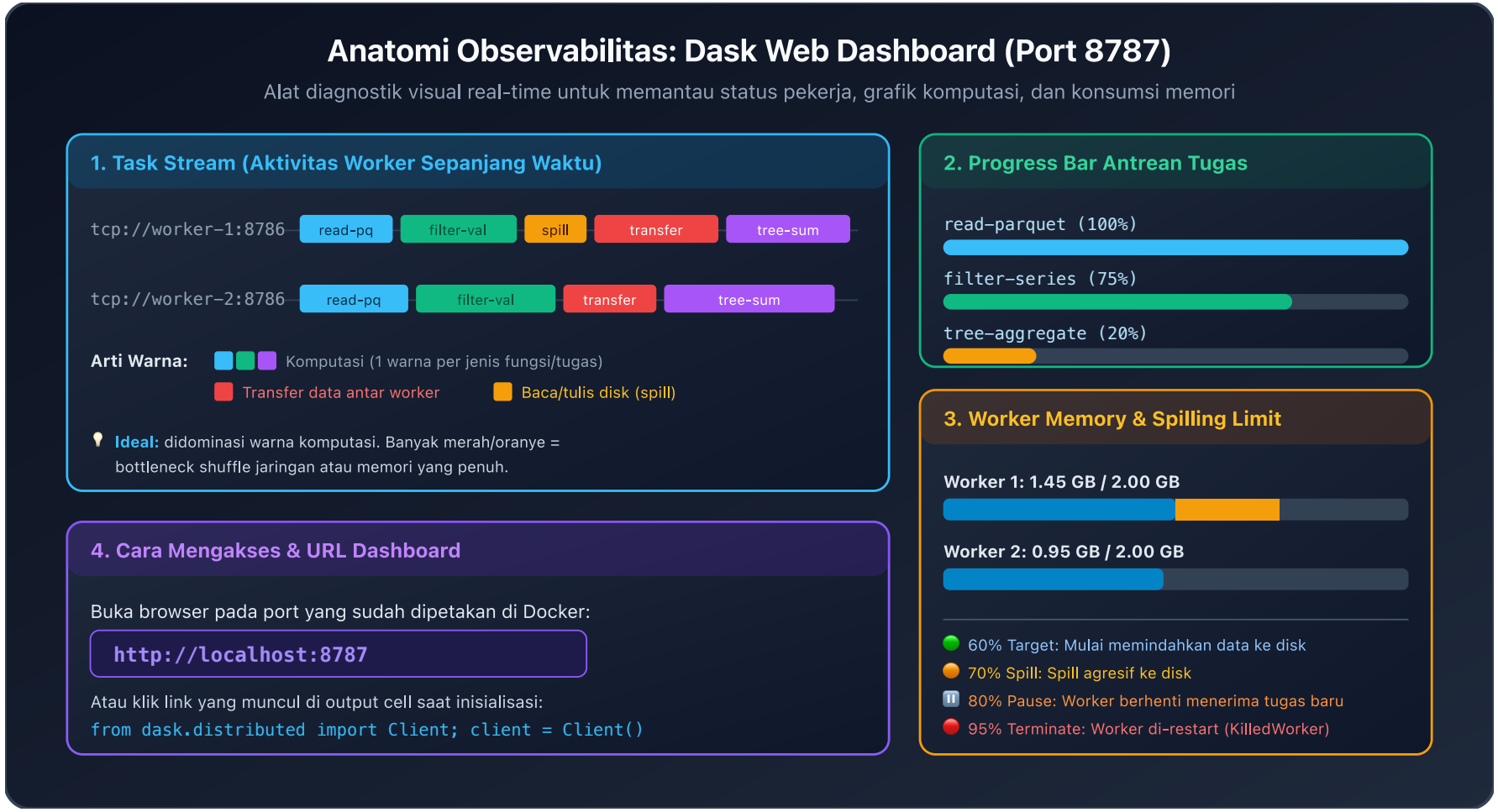
Metode: .persist()
Mengeksekusi Task Graph di latar belakang dan menyimpan partisi hasil **tetap berada di memori pekerja kluster (RAM Workers)** sebagai Dask DataFrame baru.

Kapan Menggunakan:

- Setelah operasi pembersihan (*cleaning*) atau *filtering* awal yang berat.
- Saat dataset yang sama akan dianalisis berulang kali dengan berbagai kueri berbeda, agar data mentah tidak perlu dibaca ulang dari disk.

Ilustrasi Visual: Menguasai Dask Web Dashboard (Port 8787)

Dask Web Dashboard adalah senjata utama data engineer untuk mendeteksi *bottleneck* performa secara langsung.



Tautan Sumber: Dask Dashboard Diagnostics Guide

Strategi Partisi & Best Practices (*Rules of Thumb*)

Efisiensi komputasi Dask 80% ditentukan oleh ketepatan penataan partisi data Anda.

Aturan Emas: Partisi 100MB – 200MB

Usahakan ukuran setiap partisi di memori RAM berada di rentang **100 MB hingga 200 MB** dalam format terdekompresi.

Mengapa Bukan 1MB (Terlalu Banyak Partisi)?

- Jika Anda memiliki 100GB data yang dibagi menjadi partisi 1MB = **100.000 partisi**.
- Scheduler Dask akan kehabisan memori hanya untuk menyimpan metadata grafik tugas, dan waktu CPU terbuang untuk *scheduling overhead*.

Mengapa Bukan 5GB (Terlalu Sedikit Partisi)?

- Jika satu partisi berukuran 5GB dimasukkan ke worker berkapasitas RAM 4GB, sistem akan langsung mengalami **Out-of-Memory (OOM)**.
- Pekerja lain akan menganggur (*idle*) karena tidak ada cukup partisi untuk dibagi rata.

Cara Memperbaiki Ukuran Partisi:

```
# Berdasarkan jumlah partisi yang diinginkan
ddf = ddf.repartition(npartitions=16)

# Berdasarkan target ukuran memori per partisi
ddf = ddf.repartition(partition_size="100MB")
```

Diagnostik Masalah Terdistribusi & Failure Modes

Bekerja dengan komputasi paralel membawa jenis kesalahan baru yang tidak pernah ditemui pada script Python biasa.

1. Error: `KilledWorker`

Penyebab paling sering kegagalan komputasi Dask. Terjadi ketika sistem operasi atau *container* membunuh proses worker karena melebihi batas alokasi memori (*Out-Of-Memory*).

Langkah Mitigasi:

1. Tambah jumlah partisi agar ukuran chunk lebih kecil (`partition_size="75MB"`).
2. Filter kolom yang tidak perlu sejak awal pembacaan (`columns=[...]`).
3. Kurangi parameter `threads_per_worker` dan tambah jumlah `n_workers` proses.

2. Gejala: Disk Spilling Lambat

Saat penggunaan memori worker mencapai ambang batas (default: 60–70%), Dask akan menulis kelebihan data sementara ke hard disk / SSD.

Dampak:

- Komputasi melambat drastis (I/O disk jauh lebih lambat dibanding RAM bus).
- Task Stream di dashboard akan dipenuhi warna **merah** (disk write).

Solusi: Tinjau kembali strategi partisi dan hindari operasi `set_index` tanpa memfilter baris terlebih dahulu.

Praktik Lab Hari Ini: Multi-File Parquet Processing

Di lab hands-on hari ini (`notebooks/P07_dask/07_komputasi_paralel_dask.ipynb`), kita akan mempraktikkan:

1. **Inisialisasi LocalCluster & Dask Distributed Client:**
   - Menghubungkan sesi notebook ke Dask Dashboard port 8787.
2. **Eksperimen Benchmarking `dask.delayed`:**
   - Menguji percepatan waktu komputasi dari eksekusi serial ke eksekusi paralel multi-core.
3. **Pembangkitan Multi-File Parquet Sintetis (~1.000.000+ Baris):**
   - Menulis data transaksi e-commerce ke 8 file parquet terpisah.
4. **Pembacaan & Transformasi Out-of-Core:**
   - Eksekusi pipeline agregasi, filtering, dan windowing secara *lazy*.
5. **Inspeksi Dashboard & Visualisasi Task Graph:**
   - Memantau *Task Stream*, konsumsi memori, dan membedah DAG.
6. **Eksperimen `.persist()` vs `.compute()`:**
   - Mengukur penghematan waktu kueri berulang hingga 10x lipat.

Check-in Milestone Kuliah: Tugas 2 Phase 1

Pertemuan 7 adalah batas pelaporan **Tugas 2 Phase 1: Perumusan Masalah & Akuisisi Data Skala Besar**.

Deliverables Wajib Phase 1:

1. **Struktur Repositori Tim:** Tim telah dibentuk di GitHub Classroom (2–3 mahasiswa per kelompok).
2. **Dokumentasi Arsitektur (`docs/architecture.md`):**
   - Deskripsi masalah domain bisnis atau analitik.
   - Sumber dataset skala besar (> 500.000 baris atau > 500MB).
   - Diagram alur data (Ingestion ➔ Cleaning ➔ Modeling ➔ Dashboard Streamlit).
3. **Desain Fitur Awal:** Kamus data dan rancangan variabel prediktif.

Target Pertemuan Berikutnya:

- **Pertemuan 8:** Analisis Teks (NLP) pada Data Besar & Check-in Milestone 3 Tugas 1.
- **Pertemuan 9:** Scalable Machine Learning Pipeline & Check-in Tugas 2 Phase 2.
- **Pertemuan 10:** Dashboard Streamlit Interaktif & Final Submission Tugas 1.

> **Konsultasi di Kelas:** Manfaatkan sisa waktu lab praktikum untuk mendiskusikan kelayakan dataset proyek tim Anda dengan dosen.

Rangkuman Materi Pertemuan 7

1. **Skalabilitas Komputasi:** Komputasi terdistribusi (*Scale-Out*) memecahkan masalah batasan fisik RAM dan keterbatasan CPU single-node.
2. **Arsitektur 4 Lapisan Dask:** Memisahkan *Collections* API, pembuatan grafik tugas (*DAG*), *Scheduler*, dan mesin eksekusi *Workers*.
3. **Kekuatan `dask.delayed`:** Memungkinkan paralelisasi fungsi loop dan algoritma Python kustom tanpa mengubah struktur kode secara radikal.
4. **Dask DataFrame Out-of-Core:** Mengorganisasi data tabular sebagai kumpulan banyak Pandas DataFrame per partisi untuk memproses dataset raksasa.
5. **Observabilitas Dashboard:** Port 8787 memberikan visibilitas real-time terhadap *Task Stream*, waktu komputasi, dan kondisi memori worker.
6. **Prinsip Efisiensi:** Jaga ukuran partisi pada rentang **100MB – 200MB**, dahulukan operasi *partition-local* sebelum *shuffling*, dan gunakan `.persist()` untuk kueri berulang.

Terima Kasih & Selamat Berpraktik!

Silakan buka notebook praktikum:

`notebooks/P07_dask/07_komputasi_paralel_dask.ipynb`

Dashboard Dask siap diakses di browser:

http://localhost:8787